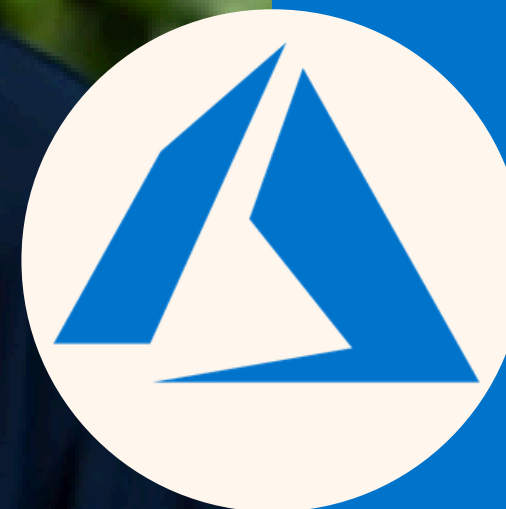


 **GET STARTED TODAY**

LUKE

GINN



AZURE
CLOUD
MASTERY



Unit 12 covered sequential vs. concurrent execution. Unit 13 covers Human-in-the-Loop – pausing agent execution for human approval before high-stakes actions.

- **Review of Unit 12:** You learned to order agent execution. Now you need to pause execution when an action requires human judgment (refunds, deletions, payments).
- **What This Unit Covers:** You will learn approval gates (pausing for human yes/no), escalation to humans, and integration with Azure Logic Apps for manual interventions.
- **Why HITL Matters:** Agents cannot make all decisions independently. High-stakes actions (deleting data, sending money) need human approval.

Exam Takeaway: Human-in-the-Loop is part of Multi-Agent Orchestration domain (35% weighting). The exam tests approval gate configuration and Logic Apps integration.



What Is Human-in-the-Loop (HITL)?

Human-in-the-Loop (HITL) is a pattern where an agent pauses its execution, requests approval from a human, and only continues if the human approves.

- **HITL Definition:** The agent reaches a point where it cannot proceed without human judgment. It stops, sends a request to a human, and waits for a response.
- **Why HITL Is Required:** For high-stakes actions – refunds over \$1000, deleting customer data, sending external communications – automatic agent action is too risky.
- **HITL vs. No HITL:** Without HITL, the agent would execute the action automatically. With HITL, the agent asks "Approve refund of \$1500 to customer X?"

Exam Takeaway: HITL pauses agent execution for human approval on high-stakes actions. The exam tests when HITL is required vs. when agents can act autonomously.



Approval Gate – The HITL Building Block

An approval gate is a pause point in agent execution where the agent waits for a human to respond with Approve or Reject before continuing.

- **Approval Gate Definition:** A checkpoint in the agent's workflow. The agent cannot pass the gate until a human provides a decision.
- **Gate Configuration:** You define the gate's trigger condition (e.g., "refund amount > \$500"), the approver(s), and the timeout period (e.g., "wait 24 hours").
- **Agent Behavior While Waiting:** The agent stops all actions for that conversation. It does not process new user messages. It only waits for the approval response.

Exam Takeaway: Approval gates pause the entire agent for that conversation until a human approves or rejects. The exam tests gate trigger conditions.



When to Use Approval Gates – High-Stakes Actions

Approval gates are required for actions with financial cost, data loss risk, legal compliance requirements, or irreversible consequences.

- **Financial Actions:** Refunds over a threshold, transferring money between accounts, purchasing subscriptions. Example: "Approve refund of \$2000 to customer?"
- **Data Destruction:** Deleting customer records, permanently removing files, clearing transaction logs. Example: "Approve deletion of user account #12345?"
- **Privileged Access:** Granting admin access, changing security settings, accessing sensitive personal data. Example: "Approve temporary admin access for support agent?"

Exam Takeaway: The exam tests your ability to identify which actions need approval gates (financial, destructive, privileged) vs. safe actions (read-only queries).



An approval request must include the action being requested, the reason, the risk level, and any supporting evidence for the approver to decide.

- **Required Fields:** `action\_description` ("Refund customer \$1500"), `reason` ("Customer claims product defective"), `risk\_level` ("High financial loss"), and `evidence` (link to order or chat log).
- **Approver Identification:** Send the request to a specific user (by email), a group (e.g., "finance-team@company.com"), or a role (e.g., "manager of customer's region").
- **Timeout Setting:** Include a deadline. Example: "Approve within 24 hours. If no response, the request will be rejected automatically."

Exam Takeaway: Approval requests must contain action, reason, risk, and evidence. The exam tests constructing complete approval messages.

Approval Response – Approve or Reject

The human approver responds with Approve (agent executes the action) or Reject (agent cancels the action and notifies the user).

- **Approve Response:** Human clicks "Approve" in email or Teams. Agent receives ``{"decision": "approve", "approved_by": "manager@company.com", "timestamp": "..."}``.
- **Reject Response:** Human clicks "Reject" and may provide a reason. Agent receives ``{"decision": "reject", "reason": "Customer already received refund last month", "rejected_by": "..."}``.
- **Agent Actions After Reject:** Agent notifies user: "Your refund request was rejected because [reason]. Please contact support." Agent ends the workflow.

Exam Takeaway: Approve = agent executes. Reject = agent cancels and notifies user. The exam tests agent behavior after each response.



Approval Timeout – What If No Response?

If the approver does not respond within the timeout period, the agent must decide whether to reject, escalate to another approver, or retry.

- **Timeout Behavior – Reject:** Default behavior. After 24 hours with no response, agent treats as reject, notifies user: "No approval received. Request cancelled."
- **Timeout Behavior – Escalate:** After first timeout (24h), escalate to a different approver or manager. Second timeout (24h) then reject.
- **Timeout Behavior – Retry:** Resend the approval request. Limited to 3 retries. After final timeout, reject. Prevents indefinite pausing.

Exam Takeaway: Timeout without response must have defined behavior (reject, escalate, or retry). The exam tests selecting appropriate timeout handling.



Azure Logic Apps is a service with over 1,400 connectors (email, Teams, Slack, SAP, Salesforce) that agents use to send approval requests to humans.

- **Logic Apps Definition:** Logic Apps is a low-code workflow engine. You build workflows visually by connecting triggers and actions from a library of connectors.
- **Why Logic Apps for HITL:** Logic Apps has pre-built connectors for email, Microsoft Teams, Slack, ServiceNow, and many human approval systems. No custom code needed.
- **Agent to Logic Apps:** Agent calls a Logic App endpoint via REST. Logic App sends the approval request via email/Teams. Agent waits for Logic App to return the decision.

Exam Takeaway: Logic Apps provides connectors to human systems. Agents call Logic Apps to send approvals. The exam tests this integration pattern.



Logic Apps includes a built-in Approval connector that sends requests to users via email or Teams and waits for Approve or Reject.

- **Approval Trigger:** In Logic App designer, add "Send approval email" action. Set approver email address, subject, body (including action details), and timeout.
- **Approval Response:** Logic App waits. When human clicks Approve or Reject, Logic App resumes and returns the decision as JSON to the caller (agent).
- **Response Available:** The Logic App output includes ``Outcome`` (Approve/Reject), ``Approver`` (email), ``Comments`` (optional reason), and ``ResponseTime``.

Exam Takeaway: Logic Apps Approval connector handles the human interaction. Agent calls Logic App, which returns the decision. No custom approval UI needed.



Your agent code calls a Logic App endpoint via REST POST with the approval details, then waits for the synchronous response containing Approve or Reject.

- **Request Body:** ``{"approver_email": "manager@company.com", "action": "Refund $1500 to customer X", "reason": "Defective product", "timeout_hours": 24}``.
- **REST Call:** ``response = requests.post(logic_app_url, headers={"Content-Type": "application/json"}, json=approval_request, timeout=60)``.
- **Wait for Response:** Logic Apps can block (wait for human response) up to the timeout. Your agent must set a long timeout (e.g., 120 seconds) or use async polling.

Exam Takeaway: The coding expectation is constructing REST calls to Logic Apps with approval details and handling the JSON response (Approve/Reject).



Asynchronous Approval – Polling Pattern

AZURE
CLOUD
MASTERY

For approvals that may take hours, your agent should not wait synchronously. Use an async pattern: call Logic App, receive a request ID, then poll for completion.

- **Step 1 – Submit Request:** Call Logic App with ``async: true`` parameter. Logic App returns ``request_id`` immediately. Agent saves the ID with the conversation.
- **Step 2 – Poll for Status:** Agent calls ``GET /approval/{request_id}`` every minute. Logic App returns ``status: pending`` until human responds.
- **Step 3 – Retrieve Result:** When human responds, Logic App returns ``status: completed, decision: approve/reject``. Agent continues execution.

Exam Takeaway: Long-running approvals (hours or days) require async polling. The exam tests the submit-get ID-poll pattern, not synchronous waiting.



Escalation Chain – Multiple Approvers

An escalation chain defines a sequence of approvers: if the first approver rejects or does not respond, the request goes to the next approver.

- **Chain Definition:** ``approvers = ["manager@company.com", "director@company.com", "vp@company.com"]``. Agent sends to first approver. If reject or timeout, move to next.
- **Escalation on Reject:** If manager rejects, agent does not go to director. Escalation only on no-response (timeout), not on explicit reject.
- **Escalation on Timeout:** Manager does not respond in 24 hours. Agent escalates to director. Director does not respond in 24 hours. Agent escalates to VP.

Exam Takeaway: Escalation chains move to the next approver only on timeout, not on reject. The exam tests escalation logic for approval workflows.



HITL in Multi-Agent Systems

In a multi-agent system, the manager agent decides when to trigger an approval gate, then pauses the entire conversation until approval completes.

- **Manager Decision:** Manager agent evaluates risk. "Refund over \$1000 requires approval." Manager calls ApprovalGate tool instead of executing refund directly.
- **Conversation Pause:** Agent Framework pauses the entire multi-agent conversation. No agent processes messages until approval completes.
- **Resume on Approval:** When approval returns (Approve), Manager calls RefundAgent. If Reject, Manager returns "Your request was rejected by [approver]."

Exam Takeaway: In multi-agent systems, the manager agent triggers approval and pauses all agents. The exam tests which agent makes the approval decision.



Foundry Trace for HITL Workflows

Foundry Trace records approval gate triggers, waiting periods, and approval decisions as spans, showing where agents paused for human input.

- **Approval Trigger Span:** Span attributes: ``gate_name``, ``action_requested``, ``approver_email``, ``timeout_hours``. Shows when agent requested approval.
- **Waiting Period Span:** Span shows ``duration`` from request to response. If timeout, span shows ``status: timeout`` and ``duration`` = timeout value.
- **Approval Decision Span:** Span attributes: ``decision`` (approve/reject), ``approver``, ``response_time_ms``. Shows how long human took to respond.

Exam Takeaway: Foundry Trace shows approval delays as waiting spans. The exam tests reading traces to identify slow approvals and timeout failures.



Approval requests must verify that the responding human is the intended approver, not an attacker impersonating them.

- **Logic Apps Authentication:** Logic App approval connector sends a unique link to the approver's email. Only that email address can click the link.
- **Entra ID Integration:** For Teams approval, Logic App checks the responder's Entra ID. Only the user matching the `approver\_email` can approve.
- **Audit Log:** Record who approved, when, and from which IP address. If approval came from unusual location, raise security alert.

Exam Takeaway: Approval systems must verify approver identity. The exam tests using unique links and Entra ID validation to prevent impersonation.



Unit 13 Summary – Human-in-the-Loop

AZURE
CLOUD

MASTERY

You have learned to use approval gates for high-stakes actions, integrate with Logic Apps for human notifications, and handle timeouts and escalations.

- **When to Use HITL:** High-stakes actions (financial, destructive, privileged). Read-only queries do not need approval.
- **Approval Gate Components:** Trigger condition, approver(s), timeout, escalation chain, and behavior on approve/reject/timeout.
- **Logic Apps Integration:** Call Logic App REST endpoint with approval details. Use async polling for long-running approvals.
- **Observability:** Foundry Trace records approval spans with decision and duration.
- **Next Unit Preview:** Unit 14 introduces Foundry Trace in depth – OpenTelemetry-based debugging for reasoning loops and multi-agent failures.

Exam Takeaway: Human-in-the-Loop is 35% of the exam (Orchestration domain). Master approval gate triggers, Logic Apps integration, timeout handling, and escalation chains.

**THANKS
FOR
WATCHING**

